

A mély és biztonságos megerősítéses tanulás

A dokumentumban a mély és a biztonságos megerősítéses tanulást a szerző egy alacsony komplexitású, Python-ban írt modellel demonstrálta. A használt adathalmaz phishing és legitim webhelyek beazonosításában segítő adathalmaz volt.

A felépített modell bemutatja, hogyan használható a megerősítéses tanulás egy bekategorizálás jellegű problémára. A választott téma és adatbázis a phishing weboldalak azonosítása. A felhasznált adathalmaz olyan webhelyjellemzőket tartalmaz, amelyek alapján bizonyos pontossággal megjósolható, hogy egy adott oldal legitim vagy káros/phishing típusú. Ilyen jellemző például az IP-cím használata a normális domainnév helyett, a nagyon hosszú URL, az URL-rövidítő szolgáltatások használata, @ karakter, a több aldomain, a HTTPS állapota, a domain regisztrációs ideje, a weboldal átlagos forgalma, a PageRank vagy a Google-indexelés meg vagy nem megléte. Ezekből a jellemzőkből a program egy állapotvektort generál, amelyet az ágens bemeneti paraméterként tud használni.

Jellemzően megerősítéses tanulás során az ágens több lépésen keresztül cselekszik egy környezetben, majd a döntéseinek hosszabb távú következményei alapján tud "tanulni". Ebben a modellben ez a feladat egyszerűsített formában jelenik meg: minden weboldal egy külön állapotot jelent, az ágens pedig egyetlen döntést hoz róla: phishing vagy megbízható. Ez a megközelítés inkább egy egylépéses megerősítéses tanulási problémának, kontextuális bandita (contextual bandit) jellegű feladatnak tekinthető. Ennek ellenére a demonstráció tudja szemléltetni, az ágens döntést hoz, jutalmat vagy büntetést kap, majd a tapasztalatai alapján javítja a döntési stratégiáját a következő alkalomra.

A program első lépésben betölti a 'Training Dataset.arff' adathalmazt. A ".arff" kiterjesztés miatt az értékek kezdetben byte formában jelennek meg, ezért a program ezeket előkészítés gyanánt egész számokká alakítja. A bemeneti változók a weboldal bináris (vagy 3 kategóriás) jellemzői, a célváltozó pedig a 'Result', avagy 'Eredmény' oszlop. A címkéket a program egyszerűbb formára alakítja: a '0' jelzés a phishing oldalt, az '1' pedig a legitim oldalt jelenteni. Ezután az adatok tanító- és tesztalmazra vannak osztva, későbbi megbízhatóság teszt megfontolásból. A tanítóhalmaz alapján tanul az ágens, a tesztalmaz pedig arra szolgál, hogy ellenőrizzük, mennyire jól képes a modell új, korábban nem látott példákra helyesen dönteni.

A neurális háló stabilabb működése érdekében a program egységesíti az input adatokat. Ez azt jelenti, hogy a jellemzők értékei egy standardizált skálára kerülnek. Ez különösen neurális hálók esetén lehet hasznos, mert a különböző nagyságrendű bemenetek lassíthatják vagy bizonytalanná tehetik a tanulási folyamatot. Bár az eredeti adathalmaz jellemzői többnyire '-1',

'0' és '1' értékeket vesznek fel, a standardizálás itt is segít abban, hogy a háló konzisztensebb bemenetet kapjon.

Mély megerősítéssel tanulás:

A mély megerősítéssel tanulás lényege, hogy a megerősítéssel tanulás döntési folyamatát neurális hálóval kombináljuk. A hagyományos megerősítéssel tanulásban az ágens gyakran táblázatos formában tárolja, hogy egy adott állapotban melyik cselekvés mennyire előnyös. Ez egyszerű környezetekben működhet, de nagyobb vagy összetettebb vektorok esetén már nem túl gyakorlatias. A phishing adathalmazban minden weboldalt 30 jellemző ír le, ezért egy táblázatos módszer helyett célszerűbb egy neurális hálót használni, amely képes általánosítani a training adatok alapján.

A programban a mély megerősítéssel tanulást egy egyszerű Q-hálózat valósítja meg. A Q-hálózat feladata, hogy minden állapothoz két értéket rendeljen: az egyik megpróbálja megbecsülni, mennyire jó döntés az oldalt phishingnek jelölni, a másik pedig azt, mennyire jó döntés az oldalt legitimnek jelölni. A háló bemenete tehát a webhely 30 jellemzője, kimenete pedig két Q-érték. Az ágens mindig azt az akciót választja, amelyhez a háló nagyobb Q-értéket rendel. Ez alól kivételt képez a tanítás korai szakasza amikor még véletlenszerű döntésekre is szükség van.

A tanítás során a program epsilon-greedy stratégiát alkalmaz. Ez azt jelenti, hogy a program eleinte gyakran véletlenszerűen választ akciót, vagyis egyszerűen feltárja a lehetséges döntési utak halmazát. Később, ahogy egyre több tapasztalatot szerez, fokozatosan csökken a véletlenszerű döntések aránya, és a program egyre inkább a megtanult Q-értékek alapján dönt. Ez a folyamat jól szemlélteti a megerősítéssel tanulás egyik alapvető dilemmáját: az ágensnek egyensúlyt kell találnia a felfedezés és a már megtanult stratégia kihasználása között.

A jutalmazási rendszer az alapmodellben egyszerű. Ha az ágens helyesen dönt, akkor '+1' jutalmat kap, ha rosszul dönt, akkor '-1' büntetést kap. Például ha egy phishing oldalt phishingnek jelöl, akkor jutalmat kap. Ha ugyanezt az oldalt legitimnek jelöli, akkor büntetést kap. Ugyanez igaz fordítva is: ha egy legitim oldalt legitimnek jelöl, az helyes döntés és '+1' jutalmat kap, ha pedig phishingnek jelöli, az hibás döntés és ennek megfelelően van „büntetve”. A modell célja, hogy a tanítás során olyan döntési stratégiát tanuljon, amely maximalizálja a kapott jutalmat.

A mély megerősítéssel tanulási modell tehát neurális háló segítségével tanulja meg, hogy a különböző weboldaljellemzők alapján melyik döntés valószínűleg előnyösebb. A megoldás azért nevezhető mély megerősítéssel tanulásnak, mert a döntési értékek becslését nem egyszerű szabályrendszer vagy táblázat végzi, hanem egy neurális háló. A háló két rejtett rétegből áll, ReLU függvényeket használ, és a kimeneten a két lehetséges akcióhoz tartozó értékeket adja vissza.

A program futtatása után az alap mélytanulósos modell a tesztalmazon körülbelül 93,49%-os pontosságot ért el. Ez azt jelenti, hogy a tesztalmazban szereplő weboldalak döntő többségét helyesen sorolta be phishing vagy legitim kategóriába. A konfúziós mátrix alapján azonban látható volt, hogy a modell 100 esetben követett el olyan hibát, amikor egy valóban phishing oldalt legitimnek jelölt. Ez a phishing felismerésnél különösen fontos, mert ez a típusú hiba közvetlen biztonsági kockázatot jelenthet a felhasználó vagy akár egy egész vállalat számára. Ha egy rendszer egy legitim oldalt tévesen phishingnek jelöl, az okozhat kellemetlenségeket, de kevésbé veszélyes. Ha viszont egy phishing oldalt legitimnek tekint, akkor a gyanútlan felhasználó megadhat érzékeny adatokat egy rosszindulatú oldalon.

Kontrasztban a biztonságos megerősítéses tanulás:

A biztonságos megerősítéses tanulás célja, hogy az ágens ne csak magas teljesítményt érjen el, hanem kerülje a veszélyes döntéseket is. Ez különösen fontos olyan területeken, ahol egy hibás döntés komoly következményekkel járhat. Példa lehet, hogy egy önvezető autó esetében nem elegendő, hogy az autó átlagosan jól vezeti el magát; a rendszernek a ritka, de veszélyes helyzeteket is kezelnie kell. Hasonló logika alkalmazható a phishing oldalak beazonosítására is. Itt nemcsak a pontosság számít, hanem az is, hogy a rendszer minél kevesebb phishing oldalt engedjen át legitimként.

A projektben a biztonságos megerősítéses tanulás két egyszerű módon jelenik meg. Az első mód a jutalmazási rendszer módosítása. Az alap mélytanulósos modell minden hibát ugyanúgy büntetett, vagyis mindegy volt, hogy a modell egy legitim oldalt jelölt phishingnek, vagy egy phishing oldalt jelölt legitimnek. A biztonságos megerősítéses tanulás verzióban viszont a phishing oldalt legitimnek jelölő hiba nagyobb büntetést kap. Ez a hiba a programban hamis negatív esetnek tekinthető, mert a veszélyes oldalt a modell nem ismeri fel. Emiatt a SRL (biztonságos megerősítéses tanulás) modell ilyen esetben nem egyszerűen '-1', hanem '-5' büntetést kap. Ezzel a modell azt tanulja meg, hogy a phishing oldalak átengedése különösen kerülendő.

A második biztonsági elem egy egyszerű safety shield, vagyis pajzs-réteg. A safety shield nem magát a neurális hálót helyettesíti, hanem egy ellenőrző réteggént működik a döntés után. A programban ez a pajzs megszámolja, hogy az adott weboldal jellemzői közül hány vesz fel gyanús, vagyis '-1' értéket. Ha elég ilyen gyanús jellemző található, és a modell mégis legitimnek jelölné az oldalt, akkor a pajzs felülírja a döntést, és phishing kategóriába átsorolja az oldalt. Ez egy nagyon egyszerű szabály, de jól szemlélteti a biztonságos megerősítéses tanulás alapelvét: bizonyos veszélyes döntéseket nem engedünk végrehajtani, még akkor sem, ha a tanult modell eredetileg azt választaná.

A biztonságos változat tehát nemcsak tanul, hanem korlátozva is van a veszélyes működés szempontjából. Ez a megközelítés közelebb áll ahhoz, ahogyan valós rendszerekben is gondolkodhatunk a mesterséges intelligencia biztonságáról. Egy modell önmagában hibázhat,

ezért hasznos lehet olyan ellenőrző rétegekkel körülvenni, amelyek a legkockázatosabb döntéseket kiszűrik. A phishing felismerésben ez azt jelenti, hogy a rendszer inkább legyen óvatosabb, és jelöljön több oldalt gyanúsnak, mintsem, hogy veszélyes phishing oldalakat engedjen át legitimként.

A biztonságos RL modell futtatása után a pontosság körülbelül 91,24% lett. Ez alacsonyabb, mint az alap RL modell eredménye. Első ránézésre ez hátránynak tűnhet, de a biztonsági szempontból fontosabb mutató jelentősen javult. A veszélyes hibák száma, vagyis azoknak az eseteknek a száma, aminek a modell egy phishing oldalt legitimnek jelölt, 100-ról 34-re csökkent. Ez azt mutatja, hogy a biztonságos modell óvatosabban működik, és sokkal kevesebb kockázatos következtetést von le.

Ez az eredmény jól mutatja a biztonságos megerősítési tanulás egyik központi kompromisszumát. A maximális pontosság és a maximális biztonság nem mindig ugyanabba az irányba mutat. Egy biztonságosabb rendszer néha több legitim oldalt is tévesen phishingnek jelölhet, vagyis nőhet a hamis pozitív hibák száma. Ugyanakkor csökkenhet azoknak a hibáknak a száma, amelyek valóban veszélyesek a felhasználóra nézve. A phishing oldalak felismerésénél ez elfogadható kompromisszum lehet, mert egy téves figyelmeztetés kevésbé súlyos, mint egy veszélyes oldal átengedése.

A bemutatott biztonságos RL megoldás tehát egyszerű, de szemléletes. Nem bonyolult matematikai korlátozásokat vagy fejlett biztonsági algoritmusokat használ, hanem két könnyen érthető elemet: nagyobb büntetést a veszélyes hibákra és egy szabályalapú védőpajzsot. Ez a megközelítés könnyen átlátható, rövid Python-kóddal megvalósítható, és az eredményeken keresztül világosan bemutatja, mit jelent a biztonság szempontjának beépítése a megerősítési tanulásba.

Összefoglalás

A mindössze cirka 200 soros Python-program a felhasznált könyvtárak importálásával kezdődik. A 'pandas' és a 'scipyio arff' az adatok betöltéséhez, a 'scikit-learn' a tanító- és tesztalmaz létrehozásához, a standardizáláshoz és a kiértékeléshez szükséges. A neurális háló megvalósításához a program a PyTorch könyvtárat használja. Ezután következik az adathalmaz betöltése, a célváltozó átalakítása, majd az adatok felosztása.

A környezetet a 'PhishingEnvironment' class reprezentálja. Ez az osztály képes véletlenszerűen kiválasztani egy weboldalt az adathalmazból, majd a modell döntése alapján jutalmat vagy büntetést adni. A környezet egyszerűsége miatt minden epizód egyetlen döntésből áll. Az ágens kap egy állapotot inputként, kiválaszt egy akciót, majd a döntés függvényében azonnal megkapja a jutalmat/büntetést. Ez a struktúra elegendő ahhoz, hogy a megerősítési tanulás alapelvét demonstrálja.

A biztonságos Q hálózatot a 'QNetwork' class valósítja meg. A háló bemenete 30 értéket vár, mert az adathalmazban 30 bemeneti jellemző van. Ezután két rejtett réteg következik, majd a kimeneti réteg két értéket ad vissza. Az első kimenet a phishing akcióhoz tartozó Q-érték, a második pedig a legitim akcióhoz tartozó Q-érték. Az ágens a nagyobb Q-értékű akciót választja.

A tanítás a 'train_agent' függvény végzi. A függvény minden iteráció folyamán kiválaszt egy véletlenszerű mintát, majd az epsilon-greedy stratégia alapján hoz döntést. A kapott jutalom alapján frissíti a neurális háló súlyértékeit. Mivel ez egy egy lépéses feladat, a célérték egyszerűen az adott döntéshez tartozó jutalom. A modell tehát azt tanulja meg, hogy melyik állapotban melyik akció vezet nagyobb jutalomhoz.

A kiértékelést a program pontossággal, konfúziós mátrixszal és egy classification reporttal végzi. A pontosság megmutatja, hogy a modell összesen hány esetben döntött helyesen. A „konfúziós mátrix” ennél részletesebb képet ad, mert külön megmutatja, hogy hány phishing oldalt ismert fel helyesen, hány phishing oldalt jelölt tévesen legitimnek, hány legitim oldalt jelölt tévesen phishingnek, és hány legitim oldalt ismert fel helyesen. (lásd képernyőképet a dokumentum alján) A modell szempontjából különösen fontos a phishing oldalak téves legitimnek jelölése, mert ez jelenti a veszélyes hibát.

Összességében vizsgált modell megmutatja, hogyan lehet egy klasszikus osztályozási problémát megerősítéses tanulási szemléletben megoldani. A Deep RL változat neurális hálót használ a döntések megtanulására, míg a biztonságos RL változat a jutalmazási rendszer és egy egyszerű védőpajzs segítségével csökkenti a veszélyes hibák számát. A modell eredménye alapján az alapmodell magasabb általános pontosságot ér el, de a biztonságos modell lényegesen kevesebb phishing oldalt enged át legitimként. Ez jól illusztrálja, hogy a biztonságos megerősítéses tanulásban nem csak az számít, hogy a modell átlagosan jól teljesítsen, hanem az is, hogy a különösen kockázatos döntéseket elkerülje.

A modellhez írt kód az alábbi github repositoryban megtalálható, a használt adathalmaz és egyéb kiegészítő fájlokkal kiegészítve. A transzparencia kedvéért leszögezendő, hogy a kód írásában, illetve helyességének ellenőrzésében az OpenAI GPT5.5 illetve a Google Gemini 3.1Pro LLM modellje is asszisztáltak.

https://github.com/MiloVrh/RL_demo

Illetve alább beillesztve is megtalálható a használt kód, alatta pedig a futtatással és 42-es Seed értékkel létrejövő kimenet.

```
#Egyszerű Python snippet mély és biztonságos megerősítéses tanulásra
#Adathalmaz: Training Dataset.arff (Phishing Websites)
#link az adathalmazhoz: https://archive.ics.uci.edu/dataset/327/phishing+websites

#Ötlet:
#- Minden weboldal egy állapot (state): 30 jellemzőből álló vektor.
#- Az ágens két akció közül választ:
#   0 = phishing oldalnak jelöli
#   1 = legitim oldalnak jelöli
#- Jutalom:
#   +1, ha jól dönt
#   -1, ha rosszul dönt
#- Biztonságos tanulás változat:
#   hamis negatív esetben (phishing oldalt legitimnek mond) nagyobb büntetést kap,
#   mert ez a felhasználó számára veszélyesebb hiba.
#   Ezen kívül egy egyszerű safety shield felülírhatja a döntést, ha sok jellemző gyanús.

import random
import numpy as np
import pandas as pd
from scipy.io import arff
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

import torch
import torch.nn as nn
import torch.optim as optim

DATA_PATH = "Training Dataset.arff"
#seed használata rekonstruálhatóságért
SEED = 42
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)

# 1. Adatok betöltése
def load_arff(path):
    data, meta = arff.loadarff(path)
    df = pd.DataFrame(data)

    # bytes -> egész érték átalakítás, mert az .arff file beolvasás után b:-1 formában jönnek elő az
    értékek
    for col in df.columns:
        df[col] = df[col].apply(lambda x: int(x.decode("utf-8")) if isinstance(x, bytes) else int(x))

    X = df.drop("Result", axis=1).values.astype(np.float32)

    # Eredeti címkék: -1 = phishing, 1 = legitim
    # A modell számára így módosítva: 0 = phishing, 1 = legitim
    y_original = df["Result"].values
    y = np.where(y_original == -1, 0, 1).astype(np.int64)

    return X, y, df

# 2. Egyszerű környezet: phishing klasszifikáció mint egy lépéses megerősítéses tanulás feladat
class PhishingEnvironment:
    def __init__(self, X, y, safe_mode=False):
        self.X = X
        self.y = y
        self.safe_mode = safe_mode

    def sample(self):
        idx = np.random.randint(0, len(self.X))
        return self.X[idx], self.y[idx]

    def reward(self, true_label, action):
```

```

# action: 0 = phishing, 1 = legitimate
if action == true_label:
    return 1.0

if self.safe_mode:
    # Veszélyesebb hiba: phishing oldalt legitimnek jelölünk.
    false_negative = (true_label == 0 and action == 1)
    if false_negative:
        return -5.0

return -1.0

# 3. Mély Q-netwrok: kis komplexitású neurális háló, dimenzió-szám input függé
class QNetwork(nn.Module):
    def __init__(self, input_dim, output_dim=2):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim, 32),
            nn.ReLU(),
            nn.Linear(32, 16),
            nn.ReLU(),
            nn.Linear(16, output_dim)
        )

    def forward(self, x):
        return self.net(x)

# 4. Tanítás: egyszerű epsilon-greedy DQN inspirált megoldás - gemini segítségével írva
def train_agent(X_train, y_train, safe_mode=False, episodes=5000, lr=0.001):
    env = PhishingEnvironment(X_train, y_train, safe_mode=safe_mode)
    model = QNetwork(input_dim=X_train.shape[1])
    optimizer = optim.Adam(model.parameters(), lr=lr)
    loss_fn = nn.MSELoss()

    epsilon = 1.0
    epsilon_min = 0.05
    epsilon_decay = 0.995

    for ep in range(episodes):
        state, true_label = env.sample()
        state_tensor = torch.tensor(state).unsqueeze(0)

        # Felfedezés vagy kihasználás
        if random.random() < epsilon:
            action = random.randint(0, 1)
        else:
            with torch.no_grad():
                q_values = model(state_tensor)
                action = int(torch.argmax(q_values).item())

        r = env.reward(true_label, action)

        q_values = model(state_tensor)
        target = q_values.clone().detach()
        target[0, action] = r

        loss = loss_fn(q_values, target)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        epsilon = max(epsilon_min, epsilon * epsilon_decay)

    return model

# 5. Safety shield: nagyon egyszerű biztonsági szabály
def safety_shield(x, proposed_action, threshold=12):
    #Ha sok feature értéke -1, akkor az oldal erősen gyanús.
    #Ilyenkor nem engedjük, hogy az ágens legitim oldalnak jelölje.
    #Ez egy egyszerű biztonságos megerősítéses tanulás jellegű védőréteg.
    suspicious_count = np.sum(x == -1)

```



```

    if suspicious_count >= threshold and proposed_action == 1:
        return 0 # felülrás: phishing

    return proposed_action

# 6. Kiértékelés
def predict(model, X, use_shield=False):
    model.eval()
    preds = []

    with torch.no_grad():
        for x in X:
            x_tensor = torch.tensor(x).unsqueeze(0)
            q_values = model(x_tensor)
            action = int(torch.argmax(q_values).item())

            if use_shield:
                action = safety_shield(x, action)

            preds.append(action)

    return np.array(preds)

def evaluate(name, y_true, y_pred):
    acc = accuracy_score(y_true, y_pred)
    cm = confusion_matrix(y_true, y_pred, labels=[0, 1])
    # cm felépítés:
    # [[phishing helyesen, phishing tévesen legitimnek],
    #  [legitim tévesen phishingnek, legitim helyesen]]
    false_negatives = cm[0, 1]

    print("\n" + "=" * 60)
    print(name)
    print("Pontosság:", round(acc, 4))
    print("Konfúziós mátrix [0=phishing, 1=legitim]:")
    print(cm)
    print("Veszélyes hibák száma, amikor phishing oldalt legitimnek mondott:", false_negatives)
    print("\nRészletes riport:")
    print(classification_report(y_true, y_pred, target_names=["phishing", "legitim"]))

# 7. Main
if __name__ == "__main__":
    X, y, df = load_arff(DATA_PATH)

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.25, random_state=SEED, stratify=y
    )

    # Skálázás: neurális hálónál stabilabb tanítást ad
    scaler = StandardScaler()
    X_train = scaler.fit_transform(X_train).astype(np.float32)
    X_test = scaler.transform(X_test).astype(np.float32)

    print("Adathalmaz mérete:", X.shape)
    print("Tanító minták:", X_train.shape[0])
    print("Teszt minták:", X_test.shape[0])
    print("Célváltozó: 0 = phishing, 1 = legitim")

    # 1) Deep RL: sima jutalmazással
    deep_rl_model = train_agent(X_train, y_train, safe_mode=False, episodes=5000)
    deep_preds = predict(deep_rl_model, X_test, use_shield=False)
    evaluate("1. Mély megerősítéses tanulás - alap DQN-szerű ágens", y_test, deep_preds)

    # 2) Safe Deep RL: veszélyes hibák nagyobb büntetése + safety shield
    safe_rl_model = train_agent(X_train, y_train, safe_mode=True, episodes=5000)
    safe_preds = predict(safe_rl_model, X_test, use_shield=True)
    evaluate("2. Biztonságos mély megerősítéses tanulás - büntetés + safety shield", y_test,
    safe_preds)

```

Output:

```

Adathalmaz mérete: (11055, 30)
Tanító minták: 8291
Teszt minták: 2764
Célváltozó: 0 = phishing, 1 = legitim
=====
1. Mély megerősítéses tanulás - alap DQN-szerű ágens
Pontosság: 0.9349
Konfúziós mátrix [0=phishing, 1=legitim]:
[[1125 100]
 [ 80 1459]]
Veszélyes hibák száma, amikor phishing oldalt legitimnek mondott: 100

Részletes riport:
      precision    recall  f1-score   support

 phishing         0.93         0.92         0.93         1225
  legitim         0.94         0.95         0.94         1539

 accuracy                   0.93         2764
 macro avg         0.93         0.93         0.93         2764
weighted avg         0.93         0.93         0.93         2764

=====
2. Biztonságos mély megerősítéses tanulás - büntetés + safety shield
Pontosság: 0.9124
Konfúziós mátrix [0=phishing, 1=legitim]:
[[1191  34]
 [ 208 1331]]
Veszélyes hibák száma, amikor phishing oldalt legitimnek mondott: 34

Részletes riport:
      precision    recall  f1-score   support

 phishing         0.85         0.97         0.91         1225
  legitim         0.98         0.86         0.92         1539

 accuracy                   0.91         2764
 macro avg         0.91         0.92         0.91         2764
weighted avg         0.92         0.91         0.91         2764

```